Experiment No. 3

Aim :

To implement NumPy, Pandas, Matplotlib & Seaborn for Machine Learning.

Theory :

1. Dataset Source

Dataset Name: Breast Cancer Dataset
Source Platform: Kaggle

Dataset Link : https://www.kaggle.com/code/chanchal24/breast-cancer-dataset/input

2. Dataset Description

The Breast Cancer dataset contains diagnostic measurements computed from digitized images of breast mass samples. The goal is to classify tumors as either malignant or benign.

Total Samples: 569
Number of Features: 30 numerical features
Target Variable: Diagnosis (M = Malignant, B = Benign)

Key Features Include:

- Radius Mean
- Texture Mean
- Perimeter Mean
- Area Mean
- Smoothness Mean
- Compactness Mean
- Concavity Mean
- Symmetry Mean
- Fractal Dimension Mean

Data Characteristics:

- The dataset is numerical and does not require categorical encoding.
- No significant missing values are present after preprocessing.
- Target variable was converted into binary form (Malignant = 1, Benign = 0).

3. Mathematical Formulation of the Algorithm

3.1 Decision Tree Mathematical Model

A Decision Tree is a supervised learning algorithm that recursively splits a dataset into smaller subsets based on feature values. The goal is to create nodes that are as pure as possible, meaning that most samples in a node belong to a single class.

Entropy

Entropy is used to measure the randomness or impurity present in a dataset.

$$\textbf{Entropy}(S) = -\Sigma \left(p_i \log_2 p_i \right)$$

where:

- S represents the dataset at a node
- p_i is the probability of class i

If entropy is high, the data is mixed. If entropy is low, the data is more homogeneous.

Information Gain

Information Gain determines which feature should be selected to split the data.

$$\textbf{IG}(S, A) = \textbf{Entropy}(S) - \Sigma \left(|S_v| / |S| \right) \times \textbf{Entropy}(S_v)$$

where:

- A is the feature used for splitting
- S_v represents the subset created after splitting

The feature with the highest Information Gain is chosen because it reduces uncertainty the most.

Gini Index

Another impurity measure used in Decision Trees is the Gini Index.

$$\textbf{Gini}(S) = 1 - \Sigma \left(p_i^2 \right)$$

Lower Gini values indicate better separation of classes.

Decision Trees continue splitting until stopping conditions are met, such as reaching maximum depth or minimum samples per node.

`

3.2 Random Forest Mathematical Model

Random Forest is an ensemble learning method that builds multiple Decision Trees and combines their predictions to improve accuracy and reduce overfitting.

Bootstrap Sampling (Bagging)

Each tree is trained on a randomly sampled subset of the original dataset.

$$\mathbf{D_i \subset D}$$

where:

- D is the original dataset
- D_i is the bootstrap sample used for training tree i

This introduces variability between trees.

Random Feature Selection

At each split, only a random subset of features is considered. This reduces correlation between trees and improves generalization.

Ensemble Prediction (Majority Voting)

The final prediction of the Random Forest model is obtained by combining predictions from all trees.

$$\mathbf{F(x) = mode \{ T_1(x), T_2(x), \dots , T_n(x) \}}$$

where:

- $T_i(x)$ represents prediction from each individual tree
- n is the number of trees

This ensemble approach reduces variance and makes the model more stable than a single Decision Tree.

4. Algorithm Limitations

Decision Tree Limitations

- Prone to overfitting if tree depth is large.
- Small changes in dataset may lead to different tree structures.
- Can create biased trees if classes are imbalanced.

Random Forest Limitations

- Higher computational cost compared to single tree.
- Reduced interpretability due to ensemble structure.
- Requires tuning of hyperparameters such as number of estimators and depth.

5. Methodology / Workflow

- Dataset Loading
  - Imported Breast Cancer dataset from Kaggle CSV file.
- Data Preprocessing
  - Removed unnecessary columns like ID.
  - Converted diagnosis labels into binary form.
  - Verified absence of missing values.
- Exploratory Data Analysis
  - Visualized class distribution.
  - Generated correlation heatmap to analyze relationships between features.
- Data Splitting
  - Dataset divided into training and testing sets using an 80:20 ratio.
- Model Training
  - Trained Decision Tree classifier.
  - Trained Random Forest classifier.
- Model Evaluation
  - Evaluated using accuracy score, classification report, and confusion matrix.
- Visualization
  - Decision Tree structure plotted.
  - Confusion matrices visualized.
  - Feature importance graph generated for Random Forest.
- Hyperparameter Tuning
  - GridSearchCV used to find optimal parameters for both models.

6. Performance Analysis

The performance of Decision Tree and Random Forest models was compared based on accuracy and classification metrics.

Observations:

- Random Forest achieved higher accuracy due to ensemble learning.
- Decision Tree provided easier interpretability but showed slight overfitting tendencies.
- Feature importance analysis indicated that radius, perimeter, and concavity-related features significantly influenced predictions.

The confusion matrix analysis showed that the models were effective in distinguishing malignant and benign cases with minimal misclassification.

7. Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV to improve model performance.

Decision Tree Parameters Tuned:

- max_depth
- criterion (gini, entropy)
- Min_samples_split

Random Forest Parameters Tuned:

- n_estimators
- max_depth
- min_samples_split

Impact of Tuning:

- Reduced overfitting by limiting tree depth.
- Improved overall accuracy and model stability.
- Tuned Random Forest produced the best generalization performance on unseen data.

Output :

[30]

```
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
```

[31]

```
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
print(confusion_matrix(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.9649122807017544
              precision    recall  f1-score   support

           0       0.96      0.99      0.97        71
           1       0.98      0.93      0.95        43

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114

[[70  1]
 [ 3 40]]
```

[21]

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

[22]

```
df = pd.read_csv("breast-cancer.csv")

print(df.head())
print(df.shape)
```

```
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  radius_worst  texture_worst  perimeter_worst  area_worst  \
0  ...         25.38          17.33           184.60      2019.0
1  ...         24.99          23.41           158.80      1956.0
2  ...         23.57          25.53           152.50      1709.0
```

[23]

```python
df.drop(['id','Unnamed: 32'], axis=1, inplace=True, errors='ignore')
```

[24]

```python
df['diagnosis'] = df['diagnosis'].astype(str).str.strip()
df['diagnosis'] = df['diagnosis'].map({'M':1, 'B':0})
```

[25]

```python
print(df['diagnosis'].unique())
print("NaN count:", df['diagnosis'].isnull().sum())
```

```
[1 0]
NaN count: 0
```

[26]

```python
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

print(X.shape)
print(y.shape)
```

```
(569, 30)
(569,)
```

[27]

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

```
Train size: (455, 30)
Test size: (114, 30)
```

[28]

```python
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)
```

[29]

```python
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
print(confusion_matrix(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.9473684210526315
              precision    recall  f1-score   support

           0       0.96      0.96      0.96        71
           1       0.93      0.93      0.93        43

    accuracy                           0.95       114
   macro avg       0.94      0.94      0.94       114
weighted avg       0.95      0.95      0.95       114

[[68  3]
 [ 3 40]]
```

[32]
2s

```python
param_dt = {
    'max_depth':[3,5,10,None],
    'criterion':['gini','entropy'],
    'min_samples_split':[2,5,10]
}

grid_dt = GridSearchCV(DecisionTreeClassifier(), param_dt, cv=5)
grid_dt.fit(X_train, y_train)

print("Best DT Params:", grid_dt.best_params_)
```

```
Best DT Params: {'criterion': 'entropy', 'max_depth': 3, 'min_samples_split': 5}
```

[33]
44s

```python
param_rf = {
    'n_estimators':[50,100,200],
    'max_depth':[None,5,10],
    'min_samples_split':[2,5]
}

grid_rf = GridSearchCV(RandomForestClassifier(), param_rf, cv=5)
grid_rf.fit(X_train, y_train)

print("Best RF Params:", grid_rf.best_params_)
```

```
Best RF Params: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 200}
```

[34]
0s

```python
best_rf = grid_rf.best_estimator_

y_pred_best = best_rf.predict(X_test)

print("Tuned RF Accuracy:", accuracy_score(y_test, y_pred_best))
```

```
Tuned RF Accuracy: 0.9649122807017544
```

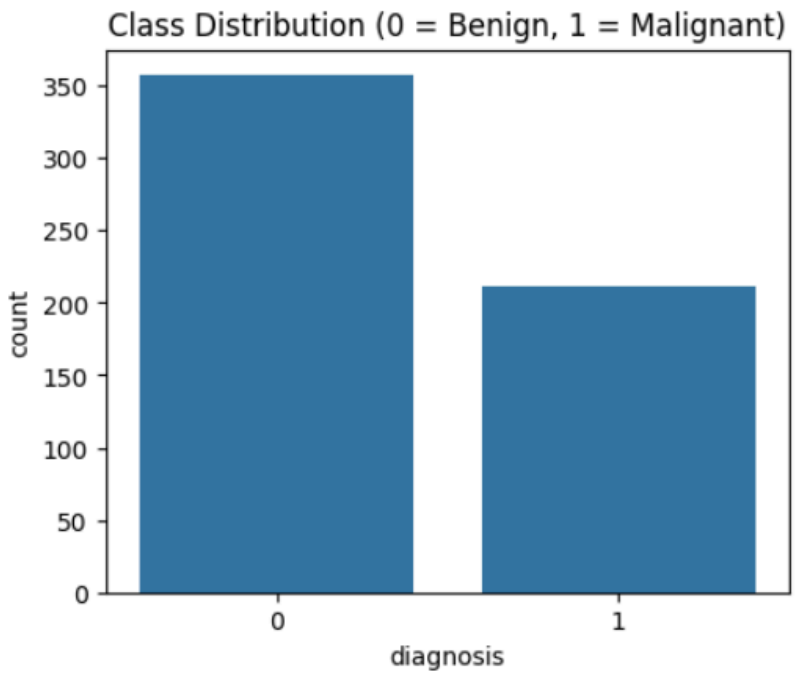
[35]
0s

```python
print("Decision Tree:", accuracy_score(y_test, y_pred_dt))
print("Random Forest:", accuracy_score(y_test, y_pred_rf))
print("Tuned Random Forest:", accuracy_score(y_test, y_pred_best))
```

```
Decision Tree: 0.9473684210526315
Random Forest: 0.9649122807017544
Tuned Random Forest: 0.9649122807017544
```

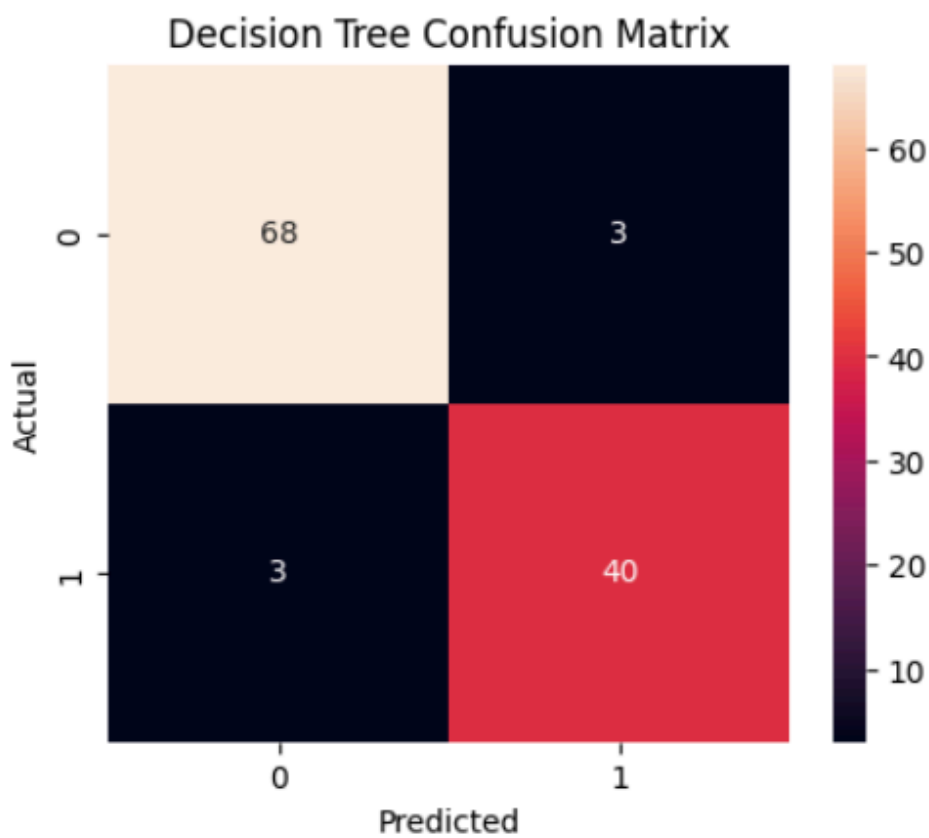
[36]
0s

```python
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
sns.countplot(x='diagnosis', data=df)
plt.title("Class Distribution (0 = Benign, 1 = Malignant)")
plt.show()
```



[40]
0s

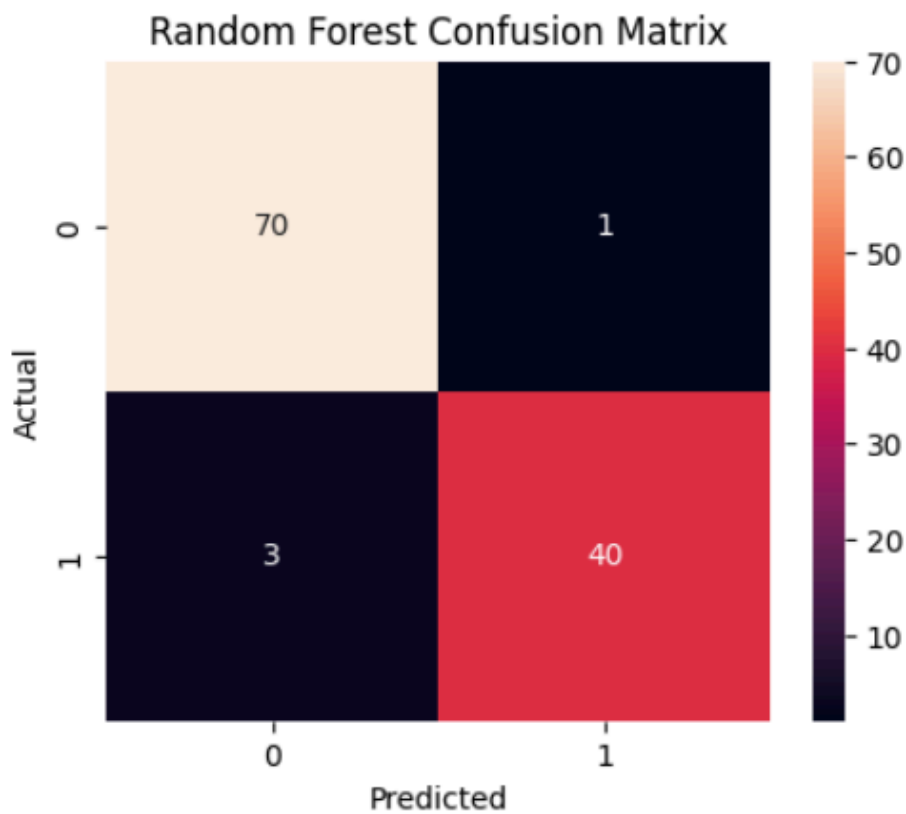
```python
cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(5,4))
sns.heatmap(cm_dt, annot=True, fmt='d')
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



[41]

0s

```
cm_rf = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(5,4))
sns.heatmap(cm_rf, annot=True, fmt='d')
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Conclusion :

The experiment successfully demonstrated classification using Decision Tree and Random Forest algorithms. Random Forest outperformed Decision Tree due to ensemble averaging, while Decision Tree provided interpretable decision rules. Hyperparameter tuning further enhanced model accuracy and reliability, making Random Forest a suitable choice for medical classification tasks.